

4. So, we have successfully added a new table to the `template1` database. Now let's try to create a new database called `dummydb` and make a list of all the tables in the `dummydb` database:

```
template1=# create database dummydb;
CREATE DATABASE
template1=# \c dummydb
You are now connected to database "dummydb" as user "postgres".
```

The `dummydb` database contains the following tables:

```
dummydb=# \dt
          List of relations
Schema |   Name   | Type  | Owner
-----+-----+-----+-----
public | dummytable | table | postgres
(1 row)
```

As expected, in the `dummydb` database, we can see the table created previously in the `template1` database.



It is important to remember that any changes made to the `template1` database will be present in all databases created after this change.

Now we will delete the `dummydb` database and the `dummy` table in the `template1` database.

Dropping tables and databases

In the next section, you will learn how to delete tables and databases. The commands we are going to learn are the following:

- `DROP TABLE`: This is used to drop a table in the database.
- `DROP DATABASE`: This is used to drop a database in the cluster.

Dropping tables

In PostgreSQL, the command needed to drop a table is simply `DROP TABLE tablename`. To do this, we have to connect to the database to which the table belongs, and then run the `DROP TABLE tablename` command.

For example, if we want to drop the `dummytable` table from the `template1` database, we have to take the following steps.

We connect to the `template1` database using the following command:

```
dummydb=# \c template1
You are now connected to database "template1" as user "postgres".
```

And we can drop the table using the following command:

```
template1=# drop table dummytable;
DROP TABLE
```

Dropping databases

In PostgreSQL, the command needed to drop a table is simply `DROP DATABASE databasename`; for example, if we want to drop the `dummydb` database, we have to execute the following command:

```
template1=# drop database dummydb ;
DROP DATABASE
```

With this, everything has now been returned to how it was at the beginning of the chapter.

Making a database copy

The following steps show you how to make a new database out of a template database:

1. Make a copy of the `forumdb` database on the same PostgreSQL cluster by performing the following command:

```
template1=# create database forumdb2 template forumdb;
CREATE DATABASE
```

By using this command, you are simply telling PostgreSQL to create a new database called `forumdb2` using the `forumdb` database as a template.

2. Connect to the `forumdb2` database as the `forum` user:

```
postgres@learn_postgresql:/$ psql -U forum forumdb2
forumdb2=>
```

3. List all the tables in the `forumdb2` database:

```
forumdb2=> \dt
      List of relations
```

Schema	Name	Type	Owner
forum	categories	table	forum
forum	j_posts_tags	table	forum
forum	posts	table	forum
forum	tags	table	forum
forum	users	table	forum

(5 rows)

You can see that the same tables that are present in the `forumdb` database are now present in this database.

Confirming the database size

We are now going to address the question of how one can determine the real size of a database. There are two methods you can use to do this: `psql` and `SQL`. Let's compare the two in the following sections.

The `psql` method

We can check the database size using the `psql` method, using the following steps:

1. First, let's connect to `forumdb` and return to expanded mode:

```
postgres@learn_postgresql:/$ psql -U forum forumdb
forumdb=> \x
Expanded display is on.
```

2. Then, execute the following command:

```
forumdb=# \l+ forumdb
List of databases
-[ RECORD 1 ]-----
Name          | forumdb
Owner         | forum
Encoding      | UTF8
Collate       | en_US.utf8
Ctype         | en_US.utf8
ICU Locale    |
Locale Provider | libc
```

```
Access privileges |
Size              | 7685 kB
Tablespace       | pg_default
Description      |
```

In the Size field, you can now see the real size of the database at that moment.

The SQL method

When using the method outlined above, you may find that you cannot connect to your database through the `psql` command. This happens when we only have web access to the database; for example, if we only have `pgadmin4` server-side installation access. If this happens, the SQL method is an alternative approach that will allow you to find the same information. To use this method, complete the following steps:

1. Execute the following command:

```
forumdb=> select pg_database_size('forumdb');
-[ RECORD 1 ]----+-----
pg_database_size | 7869231
```

The `pg_database_size(name)` function returns the disk space used by the database called `forumdb`. This means that the result is the number of bytes used by the database.

2. If you wanted a more readable result in “human” terms, you could use the `pg_size_pretty` function and write the following:

```
forumdb=> select pg_size_pretty(pg_database_size('forumdb'));
-[ RECORD 1 ]--+-
pg_size_pretty  | 7685 kB
```

As you can see, both methods give the same result.

Behind the scenes of database creation

We have just learned what commands are used to create a new database, but what happens behind the scenes when a database is created?

In this section, we will see the relationships that exist between what we perform at the SQL level and what happens physically in the filesystem; note that the `oid` numbers we see below are related to the Docker image created. The numerical values of your Docker image could be different.

To understand this, we need to introduce the `pg_database` system table:

1. Go back to the expanded mode and execute the following:

```

forumdb=> select * from pg_database where datname='forumdb';
-[ RECORD 1 ]--+-
oid           | 16386
datname       | forumdb
datdba        | 16385
encoding      | 6
datlocprovider | c
datistemplate | f
dataallowconn | t
datconnlimit  | -1
datfrozenxid  | 717
datminmxid    | 1
dattablespace | 1663
datcollate    | en_US.utf8
datctype      | en_US.utf8
daticulocale  |
datcollversion | 2.31
datacl        |

```

This query gives us all the information about the `forumdb` database. The first field is an **Object Identifier (OID)**, which is a number that uniquely identifies the database called `forumdb`.

2. Exit the `psql` environment and go to the `$PGDATA` directory (as shown in previous chapters). In a Linux Debian environment, we have to execute the following:

```
cd /var/lib/postgresql/16/main/
```

For the Docker image, the path is as follows:

```
cd /postgres/16/data
```

If we don't know what the value of `$PGDATA` is, we can execute the following as a superuser:

```

forumdb=# show data_directory;
data_directory
-----

```

```
/postgres/16/data  
(1 row)
```

3. Use the `ls` command to see what is inside the main or data (Docker image) directory:

```
postgres@learn_postgresql:~/data$ ls -l  
total 128  
drwx----- 8 postgres postgres 4096 Jan  3 09:49 base  
drwx----- 2 postgres postgres 4096 Jan  3 09:49 global  
[...]
```

As you can see, the first directory is called `base`. It contains all the databases that are in the cluster.

4. Go inside the `base` directory in order to see the contents:

```
postgres@learn_postgresql:~/data$ cd base  
postgres@learn_postgresql:~/data/base$
```

5. List all files that are present in the directory:

```
postgres@learn_postgresql:~/data/base$ ls -l  
total 40  
drwx----- 2 postgres postgres 4096 Jan  3 09:45 1  
drwx----- 2 postgres postgres 12288 Jan  3 09:14 16386  
[...]
```

As you can see, there is a directory called `16386`; its name is exactly the same as the OID in the `pg_database` catalog.



When PostgreSQL creates a new database, it copies the directory relative to the `template1` database and then gives it a new name. In PostgreSQL, databases are directories.

In this section, we have learned how to manage databases. In the next section, we will learn how to manage tables.

Managing tables

In this section, we will learn how to manage tables in a database.

PostgreSQL has three types of tables:

- **Temporary tables:** Very fast tables, visible only to the user who created them
- **Unlogged tables:** Very fast tables to be used as support tables common to all users
- **Logged tables:** Regular tables

We will now use the following steps to create a user table from scratch:

1. Let's connect to forumdb as the forum user:

```
postgres@learn_postgresql:~$ psql -U forum forumdb
forumdb=>
```

2. Execute the following command:

```
forumdb=> CREATE TABLE myusers (
  pk int GENERATED ALWAYS AS IDENTITY
  , username text NOT NULL
  , gecos text
  , email text NOT NULL
  , PRIMARY KEY( pk )
  , UNIQUE ( username )
);
CREATE TABLE
```

The `CREATE TABLE` command creates a new table. The `GENERATED AS IDENTITY` command automatically assigns a unique value to a column.

3. Observe what was created on the database using the `\d` command:

```
forumdb=> \d myusers

              Table "forum.myusers"
  Column  | Type   | Collation | Nullable |          Default
-----+-----+-----+-----+-----
pk        | integer |           | not null | generated always as
identity
username  | text    |           | not null |
```

```

gecos      | text      |          |          |
email      | text      |          | not null |
Indexes:
    "myusers_pkey" PRIMARY KEY, btree (pk)
    "myusers_username_key" UNIQUE CONSTRAINT, btree (username)

```

Something to note is that PostgreSQL has created a unique index. Later in this book, we will analyze indexes in more detail and address what they are, what kinds of indexes exist, and how to use them. For now, we will simply say that a unique index is an index that does not allow the insertion of duplicate values for the field where the index was created.



In PostgreSQL, primary keys are implemented using unique indexes.

4. Use the following command to drop a table:

```

forumdb=> drop table myusers ;
DROP TABLE

```

The preceding command simply drops the table users. The CREATE TABLE command, as we've seen before, has some useful options:

- IF NOT EXISTS
- TEMP
- UNLOGGED

We'll cover each of these in the following subsections.

The EXISTS option

The EXISTS option can be used in conjunction with entity create or drop commands to check whether the object already exists. An example of its use may be combined with the CREATE TABLE or CREATE DATABASE command. We can also use this option when we create or drop sequences, indices, roles, and schemas.

The use case is very simple – the create or drop command is executed if the EXISTS clause is true; for example, if we want to create a table named users, if the table exists, we have to execute this SQL statement:

```

forumdb=> create table if not exists users (

```

```
pk int GENERATED ALWAYS AS IDENTITY
,username text NOT NULL
,gecos text
,email text NOT NULL
,PRIMARY KEY( pk )
,UNIQUE ( username )
);
NOTICE: relation "users" already exists, skipping
CREATE TABLE
```

The command described above will only create the users table if it does not exist already; otherwise, the command will be skipped. The DROP command works similarly; the DROP TABLE command is used to drop tables. The if exists option also exists for the DROP table command; for example, if we want to drop the myusers table if it exists, we have to execute the following:

```
forumdb=> drop table if exists myusers;
NOTICE: table "myusers" does not exist, skipping
DROP TABLE
```

You can see that the command is skipped because the table does not exist. This option can be useful because if the table does not exist, PostgreSQL does not block any other subsequent instructions.

Managing temporary tables

Later in this book, we will explore sessions, transactions, and concurrency in more depth. For now, you simply need to know that a session is a set of transactions, each session is isolated, and that a transaction is isolated from everything else. In other words, anything that happens inside the transaction cannot be seen from outside the transaction until the transaction ends. Due to this, we might need to create a data structure that is visible only within the transaction that is running. In order to do this, we have to use the temp option.

We will now explore two possibilities. The first possibility is that we could have a table visible only in the session where it was created. The second is that we might have a table visible in the same transaction where it was created.

The following is an example of the first possibility where there is a table visible within the session:

```
forumdb=> create temp table if not exists temp_users (
    pk int GENERATED ALWAYS AS IDENTITY
    ,username text NOT NULL
```

```
,gecos text
,email text NOT NULL
,PRIMARY KEY( pk )
,UNIQUE ( username )
);
CREATE TABLE
```

The preceding command will create the `temp_users` table, which will only be visible within the session where the table was created.

If instead we wanted to have a table visible only within our transaction, then we would have to add the `on commit drop` options. To do this, we would have to do the following:

1. Start a new transaction.
2. Create the `temp_users` table.
3. Commit or roll back the transaction started in *Step 1*.

Let's start with *Step 1*:

1. Start the transaction with the following code:

```
forumdb=> begin work;
BEGIN
forumdb=*>
```

The `*` symbol means that we are inside a transaction block.

2. Create a table visible only inside the transaction:

```
forumdb=*> create temp table if not exists temp_users_transaction (
  pk int GENERATED ALWAYS AS IDENTITY
  ,username text NOT NULL
  ,gecos text
  ,email text NOT NULL
  ,PRIMARY KEY( pk )
  ,UNIQUE ( username )
) on commit drop;
CREATE TABLE
```

Now check that the table is present inside the transaction and not outside the transaction:

```
forumdb=*> \d temp_users_transaction
```